Practice 3
Comunicación Cliente–Servidor usando Sockets en C++

Sistemas operativos

09/05/2026

Autores :

Naela Khaldi
Zineb Hamman

Curso Académico: 2025/2026

INDICE :

1. Introduccion : 3
2. Objectivos : 3
3. Marco Teórico : 4
4. Diseño y Metodología : 5
5. Implementación : 6
6. Resultados y Pruebas : 7
7. Discusión : 8
8. Conclusión : 9
9. Manual de Ejecución : 10
10. Repositorio Git : 11
11. Bibliografía : 12

1. Introduccion :

En entornos industriales modernos (Industria 4.0), la monitorización remota mediante sensores es fundamental para el control y mantenimiento de la maquinaria. Este ejercicio simula un escenario real de monitorización industrial en el que un servidor transmite datos de temperatura a un cliente desarrollado en C++ mediante sockets TCP/IP.

El sistema de recopilación de datos requería un cliente capaz de conectarse a un servidor Python que simulaba una cámara térmica industrial, recibir los 768 valores de temperatura enviados, almacenarlos en memoria y guardarlos correctamente en un archivo local.

Para resolver este problema, se desarrolló un programa en C++, estructurado en seis funciones independientes, siguiendo el modelo cliente-servidor, junto con un script Bash que automatiza la compilación y ejecución del sistema.

2. Objectivos :

Los objetivos principales de este ejercicio son:

- Comprender el modelo cliente-servidor
- Crear y configurar correctamente un socket TCP/IP en C++
- Establecer una conexión con un servidor remoto usando `connect()`
- Recibir datos de sensores remotos usando `recv()` en un bucle
- Almacenar los datos recibidos en estructuras dinámicas (vectores)
- Guardar los datos en un archivo de salida, **datos.txt** usando `ofstream`
- Implementar el manejo básico de errores en la comunicación
- Estructurar el programa usando funciones personalizadas (modularización)
- Automatizar la compilación y ejecución usando un script Bash

3. Marco Teórico :

Sockets TCP/IP:
Un socket es un mecanismo de comunicación que permite el intercambio de datos entre procesos, ya sea dentro del mismo sistema o entre diferentes ordenadores conectados a una red. En este ejercicio, se utilizan sockets TCP/IP, que garantizan que los datos lleguen correctamente, en orden y sin pérdidas.

Proceso Cliente-Servidor:
El modelo cliente-servidor es una arquitectura en la que un programa cliente establece una conexión con un programa servidor mediante una dirección IP y un número de puerto. El servidor Python simula una cámara térmica que genera y envía 768 valores de temperatura. El cliente C++ los recibe y almacena.

`recv()` y Fragmentación TCP:
TCP puede dividir los datos en varios paquetes durante la transmisión. Por lo tanto, es necesario llamar a `recv()` en un bucle hasta que se hayan recibido todos los bytes esperados, acumulando el total recibido.

Vectores en C++:
Un `vector<unsigned char>` es un array dinámico que puede almacenar una secuencia de bytes. A diferencia de los arrays estáticos, su tamaño puede ajustarse en tiempo de ejecución. En esta práctica se usa para almacenar los 768 bytes recibidos del servidor.

Lectura y escritura de archivos (`ifstream` / `ofstream`):
La clase `ofstream` permite escribir datos en un archivo de texto, de forma similar a como "`cout`" escribe en la pantalla. El archivo se abre, se escriben los datos y luego se cierra.

Modularización mediante funciones:
Dividir el programa en funciones independientes, cada una con una responsabilidad específica,
facilita la lectura, el mantenimiento y la detección de errores. Este es uno de los criterios evaluados en esta práctica.

4. Diseño y Metodología :

El sistema consta de tres partes principales:

Servidor Python (ServidorCamara.py):

Simula el comportamiento de una cámara térmica industrial. Genera un array de 768 bytes con valores calculados como `i % 256`, lo empaqueta con `struct.pack` y lo envía al cliente mediante una conexión TCP/IP.

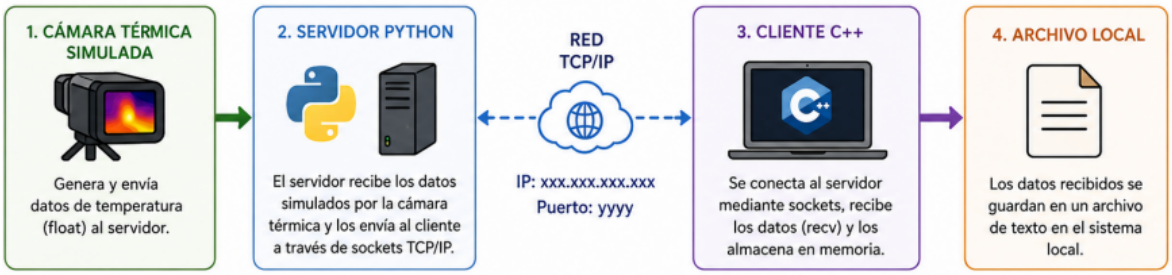
Cliente C++ (cliente.cpp):

El programa cliente se encarga de todo el procesamiento. Crea el socket, configura la dirección del servidor, establece la conexión, recibe los 768 bytes en un bucle, los almacena en un vector, los guarda en un archivo de texto y los imprime por pantalla. Está estructurado en seis funciones propias.

Script Bash (cliente.sh):

Automatiza el proceso completo: lanza el servidor Python en segundo plano, espera 2 segundos para que esté listo, compila el cliente C++ con `g++` y ejecuta el binario resultante.

Arquitectura del sistema:



5. Implementación :

Funciones C++:

- **crearSocket()**

Crea un socket TCP/IP mediante la llamada al sistema `socket(AF_INET, SOCK_STREAM, 0)`. Si la creación falla devuelve -1 y el programa termina con un mensaje de error. En caso de éxito devuelve el descriptor del socket.

- **configurarServidor()**

Rellena la estructura `sockaddr_in` con la familia de direcciones (AF_INET), el puerto convertido con `htons()` y la IP convertida de texto a binario con `inet_pton()`.

- **conectarServidor()**

Establece la conexión TCP con el servidor mediante `connect()`. Si falla, muestra un error y termina el programa. Si tiene éxito, confirma la conexión por pantalla.

- **recibirDatos()**

Recibe exactamente 768 bytes desde el servidor usando un bucle con `recv()`. El bucle acumula los bytes recibidos en cada llamada hasta completar los 768, ya que TCP puede fragmentar los datos en varios paquetes.

- **guardarArchivo()**

Abre el archivo datos.txt con `ofstream` y escribe cada byte recibido como número entero en una línea separada. Cierra el archivo al finalizar.

- **imprimirDatos()**

Recorre el vector y muestra todos los valores por pantalla en formato tabla de 16 valores por fila, usando tabulaciones para alinear la salida.

- **main()**

Llama a todas las funciones en el orden correcto: crear socket, configurar servidor, conectar, recibir datos, guardar archivo, imprimir datos y cerrar el socket. Muestra una cabecera al inicio y un mensaje de finalización al término.

Script Bash:

El script `cliente.sh` automatiza el proceso completo. Primero lanza el servidor Python en segundo plano con `python3 ServidorCamara.py` &. Espera 2 segundos con `sleep 2` para garantizar que el servidor esté listo. A continuación compila el cliente con `g++ cliente.cpp -o cliente`. Si la compilación falla, muestra un error y detiene la ejecución. Si tiene éxito, ejecuta el cliente con `./cliente`.

6. Resultados y Pruebas :

Para verificar el correcto funcionamiento del sistema, se ejecutó el script principal con el comando: sh cliente.sh

Durante la ejecución, el sistema realiza correctamente las siguientes acciones:

- Lanza el servidor Python y espera a que esté listo
- Compila el código C++ con `g++`
- Crea el socket TCP/IP y configura la dirección del servidor
- Establece la conexión con el servidor
- Recibe los 768 bytes de temperatura en bucle con `recv()`
- Almacena los datos en un `vector<unsigned char>`
- Guarda todos los valores en el archivo **datos.txt**
- Los valores recibidos se muestran por pantalla
- Cierra el socket correctamente

Salida por pantalla obtenida durante la ejecución:

```
Iniciando servidor Python...
Servidor escuchando en 127.0.0.1:12345
Compilando cliente C++...
Ejecutando cliente...
========================================
  Cliente TCP - Sensor de Temperatura
========================================

[OK] Socket creado correctamente.
[OK] Servidor configurado -> 127.0.0.1:12345
[OK] conexion establecida correctamente
Conexión aceptada desde ('127.0.0.1', 40814)
Array de datos enviado al cliente.
[OK] Datos recibidos: 768 bytes.
[OK] Datos guardados en: datos.txt

=== Valores recibidos (768 bytes) ===
0
1
2
3
4
5
6
7
8
9
```

```
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
[OK] Conexion cerrada. Programa finalizado.
```

7. Discusión :

El sistema funciona correctamente y cumple con todos los requisitos de la descripción de la práctica.

El programa en C++ utiliza correctamente el vector dinámico para almacenar los bytes recibidos. El bucle en `recibirDatos()` garantiza la recepción completa incluso si TCP fragmenta los datos. El manejo de errores cubre los casos más comunes: fallo al crear el socket, IP inválida, error de conexión y fallo en la recepción.

El script Bash automatiza con éxito todo el proceso de compilación y ejecución, incluyendo el lanzamiento del servidor Python y la espera necesaria antes de arrancar el cliente.

Cabe mencionar algunas limitaciones. La dirección IP del servidor está fijada en el código fuente y debe modificarse manualmente antes de cada ejecución. El número de bytes esperados (768) también está definido como constante, por lo que si el servidor enviará una cantidad distinta habría que actualizar el código.

Entre las posibles mejoras se incluyen: parametrizar la IP y el puerto como argumentos de línea de comandos, añadir un timestamp a cada valor recibido en el archivo de salida, y extender el cliente para recibir datos de forma continua en lugar de una sola sesión.

8. Conclusión :

Esta práctica demuestra cómo usar C++ y Bash conjuntamente para implementar un sistema de comunicación cliente–servidor en un entorno de monitorización industrial simulado. Se han aprendido varios conceptos importantes del curso.

Se aprendió a crear y gestionar sockets TCP/IP desde C++. Se comprendió el modelo cliente–servidor y el papel de la dirección IP y el puerto en la identificación de los extremos de la comunicación. Se practicó el uso de `recv()` en bucle para garantizar la recepción completa de datos fragmentados por TCP. Se trabajó con `ofstream` para la escritura de archivos y con vectores dinámicos para el almacenamiento en memoria. Finalmente, se implementó la modularización del programa en funciones propias y la automatización del flujo completo mediante un script Bash.

Se lograron todos los objetivos planteados al inicio. El cliente C++ se conecta correctamente al servidor Python, recibe los 768 bytes de temperatura, los almacena en memoria, los guarda en un archivo de texto y los muestra por pantalla. El script Bash automatiza la compilación y ejecución del sistema completo. El sistema funciona según lo previsto.

9. Manual de Ejecución :

Requisitos:

- Linux (Ubuntu 22.04 o similar)
- Compilador g++
- Python 3
- Ambos equipos en la misma red (o usar 127.0.0.1 en la misma máquina)

Pasos de configuración:
Primero, comprueba si g++ está instalado ejecutando `g++ --version` en la terminal. Si no está instalado, puedes instalarlo con `sudo apt install g++`. A continuación, abre el archivo **ServidorCamara.py** y sustituye `'Direccion-IP-del-PC'` por la IP real de tu máquina. Haz lo mismo en **cliente.cpp** en la función `configurarServidor()`.

Cómo ejecutar con el script Bash:

```
chmod +x cliente.sh
sh cliente.sh
```

Cómo ejecutar manualmente (dos terminales):
Terminal 1 — Servidor Python:

```
python3 ServidorCamara.py
```

Terminal 2 — Cliente C++:

```
g++ cliente.cpp -o cliente
./cliente
```

Qué esperar:
El script compila el programa y mostrará un mensaje de éxito. A continuación ejecutará el cliente y se mostrará la confirmación de cada paso. Los 768 valores aparecerán en pantalla y se generará el archivo **datos.txt** en la misma carpeta.

Archivos del proyecto:
La carpeta del proyecto debe contener los archivos de código fuente C++ (**cliente.cpp**), el servidor Python (**ServidorCamara.py**), el script Bash (**cliente.sh**) y el archivo de salida datos.txt que se genera al ejecutar el programa.

10. Repositorio Git :

Repository Link:

https://github.com/naelakh/Practica_3_SO.git
https://github.com/thecoolestray/Practica_3_SO.git

```
Practica_3_SO/
├── cliente.cpp
├── ServidorCamara.py
├── cliente.sh
├── datos.txt
└── README.md
```

Explicación del contenido:

- cliente.cpp:

Archivo fuente principal en C++ que contiene toda la lógica del programa. Implementa las seis funciones propias: `crearSocket()`, `configurarServidor()`, `conectarServidor()`, `recibirDatos()`, `guardarArchivo()` e `imprimirDatos()`, así como la función `main()` que las llama en el orden correcto.

- **ServidorCamara.py:**

Servidor Python que simula una cámara térmica industrial. Genera 768 valores de temperatura, los empaqueta como bytes y los envía al cliente por TCP/IP.

- **cliente.sh:**

Script Bash que automatiza la ejecución completa: lanza el servidor Python, espera 2 segundos, compila el cliente C++ y lo ejecuta

- **datos.txt:**

Archivo de salida generado automáticamente por el programa. Contiene los 768 valores de temperatura recibidos, uno por línea, en formato entero.

- **README.md:**

Archivo de documentación del proyecto. Explica la estructura del repositorio, los requisitos del sistema y las instrucciones para ejecutar el programa.

11. Bibliografía :

- Universidad de Burgos. Material de la asignatura Sistemas Operativos en UBUVirtual.
- GitHub. Documentación sobre estructura de README, informe y repositorio: https://github.com/jarain78/Practica_0_SO.git
- Wikipedia. Network socket: https://en.wikipedia.org/wiki/Network_socket